

2.3 Loop Syntax

Sue's little brother is learning his multiplication facts and has asked her to write them on a sheet of paper. Rather than spend a few minutes to hand-write the table, she decides to write a program instead. This program will need to count from 1 to 10, so a FOR loop is the obvious choice.

Objectives

By the end of this class, you will be able to:

- Demonstrate the correct syntax for a WHILE, DO-WHILE, and FOR loop.
- Create a loop to solve a simple problem.

Prerequisites

Before reading this section, please make sure you are able to:

- Convert a logic problem into a Boolean expression (Chapter 1.5).
- Generate the pseudocode corresponding to a C++ function (Chapter 2.2).

Overview

This is the first of a three part series on how to use loops to solve programming problems. The first part will focus on the mechanism of loops, namely the syntax.

Loops are mechanisms to allow a program to execute the same section of code more than once. This is an important tool for cases when an operation needs to happen repeatedly, when counting is required to solve a problem, and when the program needs to wait for an event to occur.

There are three types of loops in C++: WHILE, DO-WHILE, and FOR:

while	do-while	for
A WHILE loop is good for repeating through a given block of code multiple times.	Same as WHILE except we always execute the body of the loop at least once.	Designed for counting, usually meaning we know where we start, where we end and what changes.
<pre> { while (x > 0) { x--; cout << x << endl; } } </pre>	<pre> { do { x--; cout << x << endl; } while (x > 0); } </pre>	<pre> { for (x = 10; x > 0; x--) { cout << x << endl; } } </pre>

WHILE

The simplest loop is the WHILE statement. The WHILE loop will continue executing the body of the loop until the controlling Boolean expression evaluates to false. The syntax is:

```
while (<Boolean expression>)  
    <body statement>;
```

As with the IF statement, we can always have more than one statement in the body of the loop by adding curly braces {}s:

```
while (<Boolean expression>)  
{  
    <body statement1>;  
    <body statement2>;  
    ...  
}
```

Observe how the body of the loop is indented three spaces exactly as the body of an IF statement is indented.



Sue's Tips

The WHILE loop keeps iterating as long as the Boolean expression evaluates to true. This may seem counter-intuitive at first. Some find it easier to think that the loop keeps iterating until the Boolean expression evaluates to true.

One way to keep this straight in your mind is to read the code as “while <condition> continue to <body>.” For example consider the following code:

```
while (input < 0)  
    cin >> input;
```

This would read “While input less-than zero, continue to prompt for new input.”

It is possible to count with a WHILE loop. In this case, you initialize your counter before the loop begins, specify the continuation-condition (what must remain true as we continue through the loop) in the Boolean expression, and specify the increment logic in the body:

```
{  
    int count = 1;  
    while (count <= 10)                // continue as long as this evaluates to true  
    {                                  // use {}s because there is more than one  
        cout << count << endl;         // statement in the body of the loop  
        count++;  
    }  
}
```

This code will display the numbers 1 through 10 on the screen, each number on its own line.

Example 2.3 – While Loop

Demo

This example will demonstrate the syntax of a simple WHILE loop.

Problem

Write a program to prompt the user for his letter grade. If the grade is not in the valid range (A, B, C, D, or F), then the program will display an error message and prompt again.

```
Please enter your letter grade: B  
Your grade is B
```

...or...

```
Please enter your letter grade: E  
Invalid grade. Please enter a letter grade {A,B,C,D,F} G  
Invalid grade. Please enter a letter grade {A,B,C,D,F} C  
Your grade is C
```

The most challenging part of this problem is the Boolean expression capturing whether the user input is in the valid range. Anything that is not an A, B, C, D, or F is classified as invalid.

Solution

```
char getGrade()  
{  
    char grade;    // the value we will be returning  
  
    // initial prompt  
    cout << "Please enter your letter grade: ";  
    cin >> grade;  
  
    // validate the value  
    while (grade != 'A' && grade != 'B' && grade != 'C' &&  
           grade != 'D' && grade != 'F')  
    {  
        cout << "Invalid grade. Please enter a letter grade {A,B,C,D,F} ";  
        cin >> grade;  
    }  
  
    // return when done  
    return grade;  
}
```

Challenge

As a challenge, modify `getGrade()` so either uppercase or lowercase letter grades are accepted. This can be done by either doubling the size of the Boolean expression to include lowercase letters or by converting the grade to uppercase if it is lowercase.

See Also

The complete solution is available at [2-3-while.cpp](#) or:

```
/home/cs124/examples/2-3-while.cpp
```



DO-WHILE

The DO-WHILE loop is the same as the WHILE loop except the controlling Boolean expression is checked after the body of the loop is executed. This is called a **trailing-condition** loop, while the WHILE loop is a **leading-condition** loop. As with the WHILE statement, the loop will continue until the controlling Boolean expression evaluates to `false`. The syntax is:

```
do
    <body statement>;
while (<Boolean expression>);
```

DO-WHILE loops are used far less frequently than the aforementioned WHILE loops. Those scenarios when the DO-WHILE loop would be the tool of choice center around the need to ensure the body of the loop gets executed at least once. In other words, it is quite possible the controlling Boolean expression in a WHILE loop will evaluate to `false` the first time through, removing the possibility the body of the loop is executed. This is guaranteed to not happen with the DO-WHILE loop because the body always gets executed *first*.

Example

Consider the following code prompting the user for his age:

```
{
    int age;
    do
    {
        cout << "What is your age? ";
        cin >> age;
    }
    while (age < 0);
}
```

// the "do" keyword on its own line
// use {}s when there is more than one
// statement in the body of the loop

// keep the {}s on their own line
// continue until this evaluates to false

In this example, we want to prompt the user for his age at least once. The code will continue prompting the user for his age as long as the user enters negative numbers. Note how the `while` part of the DO-WHILE loop is on a separate line from the `{}`s. This is because the style guide specifies that `{}`s must be on their own lines.

Sam's Corner



It turns out that the WHILE loop and the DO-WHILE loop solve exactly the same set of problems. Any DO-WHILE loop can be converted to a WHILE loop by bringing one instance of the body outside the loop. In the above example, the equivalent WHILE loop would be:

```
{
    int age;
    cout << "What is your age? ";
    cin >> age;

    while (age < 0)
    {
        cout << "What is your age? ";
        cin >> age;
    }
}
```

// one copy of the body outside the loop

// same condition as the DO-WHILE

// second copy in the body of the loop

Note how redundant it is to have the second copy of the body outside the loop. This is the primary advantage of the DO-WHILE: to reduce the redundancy.

Example 2.3 – Do-while Loop

Demo

This example will demonstrate how to use a DO-WHILE loop to validate user input.

Problem

Write a program to sum the numbers the user entered. The program will continue to prompt the user until the value zero (0) is typed.

Please enter a collection of integer values. When
you are done, enter zero (0).

```
> 10
> 15
> -7
> 0
The sum is: 18
```

Solution

Because the user needs to be prompted at least once, a DO-WHILE loop is the right tool for the job.

```
int promptForNumbers()
{
    // display instructions
    cout << "Please enter a collection of integer values. When\n"
         << "\tyou are done, enter zero (0).\n";
    int sum = 0;
    int value;

    // perform the loop
    do
    {
        // prompt for value
        cout << "> ";
        cin >> value;

        // add value to sum
        sum += value;
    }
    while (value != 0);
    // continue until the user enters zero

    // return and report
    return sum;
}
```

Challenge

As a challenge, can you modify the above function so the value zero is not added to the sum?

To take this one step further, modify the above problem so -1, not 0, is the terminating condition. This will require you to modify the instructions as well.

See Also

The complete solution is available at [2-3-doWhile.cpp](#) or:

/home/cs124/examples/2-3-dowhile.cpp



FOR

The final loop is designed for counting. The syntax is:

```
for (<initialization statement>; <Boolean expression>; <increment statement>)
    <body statement>;
```

Here the syntax is quite a bit more complex than its WHILE and DO-WHILE brethren.

```
for (int count = 0; count < 5; count++)
    cout << count << endl;
```

Initialization:

The first statement to be executed in a loop.

- Can be any statement.
- We can declare and initialize a variable inside the loop:

```
for (int i = 0; ...
```

- We can initialize more than one variable that is already defined elsewhere in the code:

```
for (j = 0, k = 0; ...
```

- We can also leave it empty:

```
for (; i < 10; i++)
```

Boolean expression:

Is executed immediately before the body of the loop.

- Can be any expression.
- As long as the expression evaluates to **true**, the loop continues:
- If it is left empty, the expression evaluates to **true**. This means it will loop forever:

```
for (i = 0; ; i++)
```

Increment:

Is executed immediately after the body of the loop.

- Can be any statement.
- Usually we put a ++ or -- here:
- You can put more than one statement here:

```
for (... ; ...; i++, j--)
```

- Can be left empty:

```
for (; i < 10; )
```

While the syntax of the FOR loop may look quite complex, it has the three things any counting problem needs: where to start (initialization), where to end (Boolean expression), and how much to count by (the increment statement). For example, a FOR loop to give a countdown from 10 to zero would be:

```
{
    // a countdown, just like what Cape Kennedy uses
    for (int countDown = 10; countDown >= 0; countDown--)
        cout << countDown << endl;
}
```



Sue's Tips

While it may seem difficult to remember what the three fields of a FOR loop (Initialization, Boolean expression, and increment) are for, there is a memory clue to help you remember. Consider the loop to count to four:

```
for (int count = 1; count < 5; count++)
    cout << count << endl;
```

This reads “For count equals one, *as long as* count is less than five, add one to count.”

Example 2.3 – For Loop

Demo

This example will demonstrate how to use a FOR loop. It will allow the user to specify each of the three components of the loop (Initialization, Boolean expression, and Increment). In many ways, this is a driver program for the FOR loop.

Problem

Write a program prompt the user the parameters for counting. The program will then display the numbers in the specified range.

```
What value do you want to start at? 4
What value do you want to end at? 14
What will you count by (example: 2s): 3
    4
    7
   10
   13
```

Solution

The following program will count from start to end.

```
int main()
{
    // start
    cout << "What value do you want to start at? ";
    int start;
    cin >> start;

    // end
    cout << "How high do you want to count? ";
    int end;
    cin >> end;

    // increment
    cout << "What will you count by (ex: 2s): ";
    int increment;
    cin >> increment;

    // count it
    for (int count = start; count <= end; count += increment)
        cout << "\t" << count << endl;

    return 0;
}
```

Notice the three parts of a FOR loop: the starting condition (`int count = start`), the continuation condition (`count <= end`), and what changes every iteration (`count += increment`). As long as the continuation condition is met, the loop will continue and the body of the loop will execute.

Challenge

As a challenge, modify the above program to ensure that $\text{start} \leq \text{end}$ if increment is positive, and $\text{start} \geq \text{end}$ if increment is negative.

See Also

The complete solution is available at [2-3-for.cpp](#) or:

```
/home/cs124/examples/2-3-for.cpp
```



Problem 1

What is the output?

```
void function(int a, int &b)
{
    a = 0;
    b = 0;
}

int main()
{
    int a = 1;
    int b = 2;

    function(a, b);

    cout << "a == " << a << '\t'
         << "b == " << b << endl;
}
```

Answer:

Please see page 65 for a hint.

Problem 2

What is the output?

```
int value = 1;

int main()
{
    int value = 2;
    cout << value;

    if (true)
    {
        int value = 3;
        cout << value;

        {
            int value = 4;
            cout << value;
        }
        cout << value;
    }
    cout << value << endl;
    return 0;
}
```

Answer:

Please see page 67 for a hint.

Problem 3

What is the output?

```
{
    int j = 10;

    for (int i = 1;
        i < 3;
        i++)
        j++;

    cout << i << endl;

    return 0;
}
```

Answer:

Please see page 161 for a hint.

Problem 4-7

Write the code to implement each of the following loops:

Count from 1 to 10

Keep asking the user for input until he enters a value other than 'q'

Display the powers of two below 2,000

Sum the multiples of seven below 100

Please see page 161 for a hint.

Problem 8

Which of the following has no syntax errors?

```
do while (true)
    cout << "Infinite loop!\n";
```

```
do
    cout << "Infinite loop!\n";
while (true);
```

```
do (true)
    cout << "Infinite loop!\n";
```

```
while (true)
    cout << "Infinite loop!\n";
do;
```

Please see page 159 for a hint.

Problem 9-10

Write a program that keeps prompting the user for a number until he inputs the number 0:

Use a WHILE loop

Use a DO-WHILE loop

Please see page 159 for a hint.

Problem 11

Write the code to count down from 100 to 10 in steps of 10:

```
100
90
80
70
60
50
40
30
20
10
```

Answer:

Please see page 162 for a hint.

Problem 12-13

Write the code to prompt the user for 5 numbers and display the sum

```
Please enter 5 numbers:
#1: 54
#2: 99
#3: 12
#4: 65
#5: 34
Sum: 264
```

Answer in pseudocode:

Answer in C++:

Please see page 161 for a hint.

Assignment 2.3

Sue's silly brother Steve has a teacher who loves to give tons of math homework. This week, the assignment is to add all the multiples of 7 that are less than 100. Last week, he had to add all the multiples of 3 that are less than 100. Sue wants to make sure that her brother gets a 100% on each assignment so she decided to write a program to validate each assignment.

Example

User input in underline.

```
What multiples are we adding? 5  
The sum of multiples of 5 less than 100 are: 950
```

Another example:

```
What multiples are we adding? 7  
The sum of multiples of 7 less than 100 are: 735
```

Assignment

Make sure you run test bed with:

```
testBed cs124/assign23 assignment23.cpp
```

Don't forget to submit your assignment with the name "Assignment 23" in the header.

Please see page 162 for a hint.